Split Binary Tree

AlgoExpert · Medium · Binary Trees

The problem

- Can removing one edge split the tree into two parts with equal sums?
- Return that half-sum, else 0.

Intuition

- We need a subtree whose sum is exactly half the total.

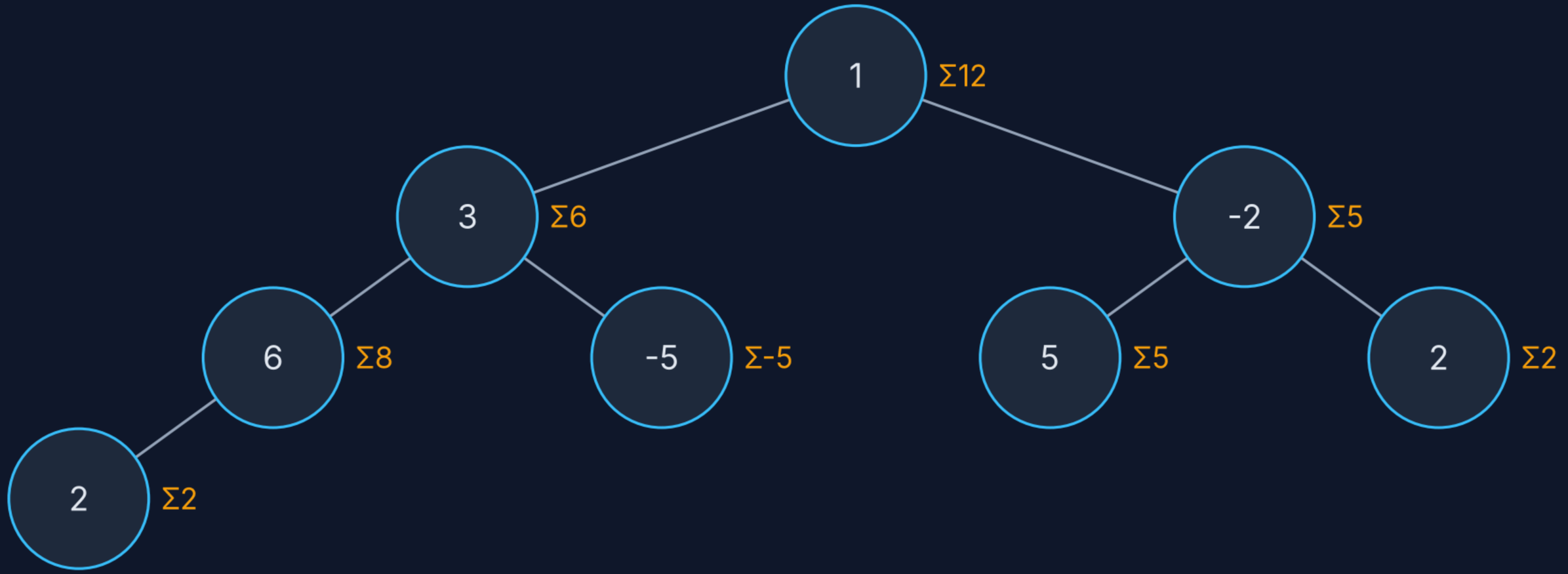
Brute force vs optimal

- Brute force: try cutting each edge, re-sum both parts → $O(n^2)$
- Two passes: total, then post-order subtree sums → $O(n)$
- Need a subtree (not the whole tree) = total / 2

Approach

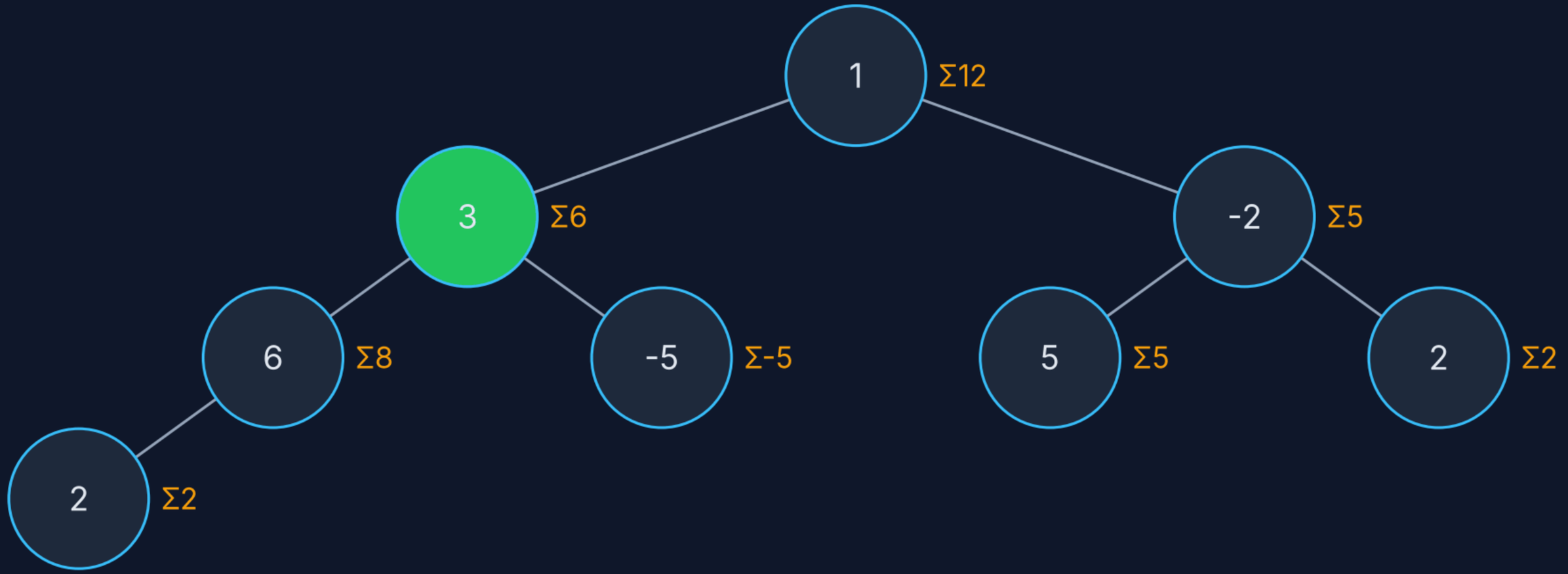
- 1. Compute the total sum
- 2. If odd, it can't split
- 3. Post-order subtree sums
- 4. Return true if any subtree equals half

Worked example



Total sum = 12 → need a subtree summing to 6

Worked example



Subtree at 3 sums to 6 → cut its edge

Java solution

```java
public class SplitBinaryTree {
    public static int splitBinaryTree(BinaryTree t) {
        int total = sum(t);
        if (total % 2 != 0) return 0;
        boolean[] found = {false};
        check(t, total / 2, found, true);
        return found[0] ? total / 2 : 0;
    }

    private static int sum(BinaryTree t) { return t == null ? 0 : t.value + sum(t.left) + sum(t.right); }

    private static int check(BinaryTree t, int half, boolean[] found, boolean root) {
        if (t == null) return 0;
        int s = t.value + check(t.left, half, found, false) + check(t.right, half, found, false);
        if (!root && s == half) found[0] = true;
        return s;
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Odd total → impossible (return 0)
- Exclude the root's own sum
- Negative values make greedy pruning unsafe

Interview tips & follow-ups

- Return the half-sum (problem's convention)
- Follow-up: maximise product of the two parts

Recap

- We need a subtree whose sum is exactly half the total.
- Complexity: O(n)